You can use `mapAsync` or `mapUnorderedAsync`, as you have seen already. This example shows how to use `mapAsync` with the connection to the database so you get to know a complete use case. Let's look for the IDs of the containers you created in chapter 9.

Each time you added cargo, you created an entry in the journal. Now, in each entry, you can look for the ID of the container. In principle, this is simple—you have to wrap the database call in a future:

```
class CPUNonBoundedSpec ... {
  ...

    val result = Source(1 to 100)          ❶
      .mapAsync(10)(each => Future(myGetPersistenceId(each)))
      .run()

  ...
}
```

❶ Looks up the first 100 entries of the journal

This example uses the same database as in section 9.4, where you created a journal. To access it, start the database by opening the chapter09b folder at the command line and running `docker-compose up`. The name of the database is `containers`, and the username and password are the same. The data is listed in table 16.1. The contents may differ because they depend on the inputs you used.

Table 16.1 Data in the `containers` database in the `event_journal` table